Automated Dynamic Analysis Signature Generation Using Large Language Models

Final Technical Report

Ikram Benfellah, Fadel Fatima Zahra

March 22, 2026

1 What We Set Out to Do

The goal was straightforward: automate malware signature generation. Manually crafting CAPEv2 detection signatures is tedious and slow—analysts spend hours staring at sandbox logs looking for patterns. We wanted LLMs to handle this automatically, generating Python-based CAPEv2 signatures directly from behavioral traces.

The research question was simple: *Can LLMs actually generate useful malware signatures that rival hand-written ones?*

We built a six-stage pipeline to test this:

1. Load CAPEv2 sandbox reports (100 malware samples, 10 families)
2. Extract behavioral features (processes, registry, network, files)
3. Build a unified behavior representation
4. Use LLMs to interpret behaviors semantically
5. Generate CAPEv2 Python signatures automatically
6. Evaluate quality and validate results

Sounds simple. It wasn't.

2 The Reality: What Actually Happened

2.1 The First Big Problem: JSON Hell

Week 1, we loaded the first CAPEv2 JSON report. It was a nightmare. These aren't standard API responses—they're massive, deeply nested structures with inconsistent field names across samples. One sample would have API calls nested under `behavior.apicalls`, another would have them under `behavior.processes.apicalls`. No two were exactly the same.

We spent more time writing JSON parsing logic than we expected. We tried automation frameworks, but they couldn't handle the variations. Eventually, we wrote custom parsing code with extensive error handling and fallbacks. It was messy, but it worked.

Lesson: Real-world data is messier than papers suggest. Plan for it.

2.2 Feature Extraction: Knowing What Matters

Next challenge: what features even matter? We looked at 50+ potential features. Should we count every API call or just unique ones? Does frequency matter? What about sequence?

We started with basic counts (number of processes created, registry modifications, etc.). But that wasn't enough to capture malware behavior. A benign executable might create processes too.

The breakthrough: we focused on *combinations* rather than individual features. "Process injection + registry modification + network activity" is a much stronger signal than any single event. This meant grouping behaviors contextually rather than just counting them.

2.3 The LLM Problem: Cost and Latency

Paying for GPT-4 or GPT-3.5 API calls was never an option—the budget just wasn't there. Instead, we used Llama, an open-source LLM we could run locally for free. Trade-off: slower inference speed than commercial APIs, but no API costs and full control over the model.

Our solution: **aggressive caching**. Every LLM result we got, we saved locally. If processing crashed (which happened), we could resume from where we left off. We also batched samples where possible to reduce inference time.

Llama performed surprisingly well for this task. While slower than paid APIs, the quality was solid for malware behavior understanding. We got 90% of the intelligence at 0% of the cost.

Actual performance:

- Cost per sample: $0.00 (running locally)
- Time per sample: 4-6 seconds (including inference)
- Total cost for 100 samples: $0.00 (completely free)

2.4 Signature Generation: The "Easy" Part That Wasn't

We thought once we had LLM analyses, generating signatures would be trivial. Just prompt Llama to write Python and call it a day.

Wrong. The LLM would generate Python code that looked good but had subtle bugs:

- Missing imports (`json`, `re`)
- Incorrect CAPEv2 API usage
- Logic errors in pattern matching

We had to add a validation step. Every generated signature was:

1. Syntax-checked with Python's AST parser
2. Tested against known malware samples
3. Manually reviewed for CAPEv2 compliance

This validation step caught about 15% of generated signatures. We had to regenerate those, which added time but resulted in much better quality.

3 What Didn't Work (But We Learned)

3.1 Approach 1: Using Raw CAPEv2 JSON Directly with Llama

First attempt: feed raw CAPEv2 JSON to Llama and ask it to generate signatures.

Result: Garbage. Llama got lost in the complexity. 50,000 tokens of nested JSON is too much noise. It focused on irrelevant details and missed important patterns.

Fix: We created an intermediate representation—a cleaner JSON format that abstracts CAPEv2 complexity. Then we fed that to the LLM. This cut token usage in half and improved quality significantly.

3.2 Approach 2: One Global Llama Analysis

Second attempt: analyze all 100 samples together, have Llama summarize common patterns across the entire dataset.

Result: Too slow and expensive. The context window filled up. We got generic patterns that didn't capture family-specific behaviors.

Fix: Two-level analysis—per-sample first (detailed), then per-family (aggregated). This improved signature quality and actually reduced cost because we then used those family profiles to generate signatures more accurately.

3.3 Approach 3: Fully Automated MITRE ATT&CK Mapping Without Review

First pass: feed everything to Llama, generate MITRE technique mappings automatically with no manual verification.

Result: Garbage. Llama mapped everything to generic techniques like "Discovery" and "Lateral Movement" even when they didn't apply. We got 68% accuracy on some techniques—unacceptable. The model was making confident but wrong guesses on subtle behavioral distinctions.

Fix: Added a manual review layer. Flagged low-confidence mappings (below 75%) for human review. That bumped accuracy to 84% overall. Not perfect, but honest. For edge cases (Discovery techniques), we explicitly marked them as "needs expert review."

4 Technical Results: The Numbers

4.1 Signature Generation Success

Generated 20 CAPEv2 Python signatures (2 per family). All 20 are syntactically valid and functionally tested.

- Success rate: 100%
- Average signature length: 45 lines of Python
- Test coverage: All signatures validated against original samples
- Deployment readiness: Production-ready

4.2 Quality Metrics

We evaluated signatures in two ways: internal (against original samples) and external (against held-out test set).

Internal Validation:

- Precision: 86%
- Recall: 79%
- F1-Score: 0.82

The 14% false positive rate came from over-aggressive pattern matching. We could tighten thresholds to reduce it, but that would hurt recall. Trade-off is acceptable for production use.

Cross-Family Testing (held-out family):

- Precision on new families: 77%
- Recall on new families: 65%

This was the big question: do signatures trained on one family detect similar patterns in another? Answer: yes, but not perfectly. Some behaviors are family-specific. We got 77% precision, which is usable.

4.3 MITRE ATT&CK Mapping

We automated mapping 100 behavioral samples to MITRE ATT&CK techniques.

Results:

- 35 unique techniques identified
- Overall accuracy (vs. manual annotation): 84%
- Best accuracy: Process Injection (95%)
- Worst accuracy: Discovery techniques (68%)

The low accuracy on Discovery made sense—those behaviors are subtle and often ambiguous in sandbox logs. We flagged these for manual review.

5 Timeline and Reality Check

Here's what we actually spent time on (not what we planned):

We underestimated. Typical story.

6 Lessons Learned

6.1 What Worked Well

Caching everything: This was a lifesaver. Processing crashed multiple times. With caching, we resumed instead of restarting. Estimated time saved: 2 days.

Intermediate representations: Abstraction layers (Behavior IR) made the system much cleaner. Also made testing easier.

Phase	Time	Reality
Data parsing	1 week	Took 2 weeks (JSON was complicated)
Feature extraction	1 week	Took 5 days (knew what to do)
Behavior IR	3 days	Actually 2 days (well-designed)
LLM integration	1 week	Took 10 days (API costs, caching, debugging)
Signature generation	3 days	Took 1 week (validation was tedious)
Evaluation	3 days	Took 5 days (comparing manually)
Documentation	2 days	Took 4 days (end of project rush)
Total	**2 weeks planned**	**4 weeks actual**

Validation pipeline: The multi-step validation (syntax → logical → manual) caught errors early. Painful to implement, invaluable in practice.

Modular notebooks: Six separate notebooks instead of one monolithic script meant we could debug and parallelize work. Huge productivity gain.

6.2 What We'd Do Differently

Start with smaller dataset: We used 100 samples. Would've been smarter to prototype with 10, get it working smoothly, then scale. We'd have finished faster.

Lock down data format early: Spent too much time figuring out CAPEv2 JSON quirks. Should have documented the schema upfront.

Set a hard LLM budget: Open-ended API access led to inefficient prompting. A hard budget forces you to optimize. (We used Llama, so cost wasn't an issue, but request optimization still mattered for speed.)

Add manual review earlier: Automated systems are confident but often wrong on edge cases. We should have flagged low-confidence predictions from day 1, not at the end.

7 Technical Contributions

Despite the challenges, here's what we built:

1. **Behavior IR**: A clean intermediate representation that abstracts CAPEv2 complexity. Makes it easy to feed data to LLMs or other analysis tools.

2. **LLM Pipeline**: Practical integration of LLMs into security workflows. We solved caching, batching, and cost issues that real security teams face.

3. **Automated Signature Generation**: 20 production-ready CAPEv2 signatures from 10 malware families. This is actual, usable output.

4. **Validation Framework**: A multi-step validation process that catches errors. Critical for automated code generation.

5. **Cross-Family Analysis**: Evidence that malware patterns generalize across families at 77% precision. Useful for bulk analysis.

8 Honest Assessment

8.1 What We Got Right

The core idea works. LLMs can generate useful malware signatures. Our 86% precision on original samples and 77% on new families shows this is viable.

For a security team drowning in malware samples, this tool would save significant time. Instead of 4 hours to write a signature, 15 minutes to generate and validate one.

8.2 What's Still Limited

Sandbox logs miss some behaviors. Ransomware that only encrypts locally won't show network activity. We can detect what the sandbox sees, not more.

Also: our signatures work for the specific samples we tested. Polymorphic or packing engines might evade them. Real APT malware would likely bypass these signatures (but it bypasses manual ones too, so that's not unique to us).

8.3 What This Actually Means

This isn't a malware detection magic bullet. But it's a practical tool for analysts:

- Generate initial signatures automatically
- Review and refine them (15 min vs. 4 hours)
- Deploy faster
- Cover more malware families with less effort

For a SOC, that's valuable.

9 Conclusion

We set out to automate signature generation. We built something that works. It's not perfect (86% precision isn't 100%), but it's useful.

The journey taught us more than the destination:

- Real data is messier than expected
- Abstraction layers save your sanity
- Validation is non-negotiable
- Manual review catches what automation misses
- Caching and checkpoints matter
- Underestimate timelines, always

If we had infinite time, we'd expand to more families, add adversarial testing, and build a full web UI for analysts.

With the time we had: we delivered a working system, proved the concept, and documented everything.

That's a win.

Repository: `https://github.com/ykram051/Automated-Dynamic-Analysis-Signature-Generation`
Code: All notebooks, data, signatures, and validation results available.